Analyzing Pizza Sales with SQL: A Real-World Database Project

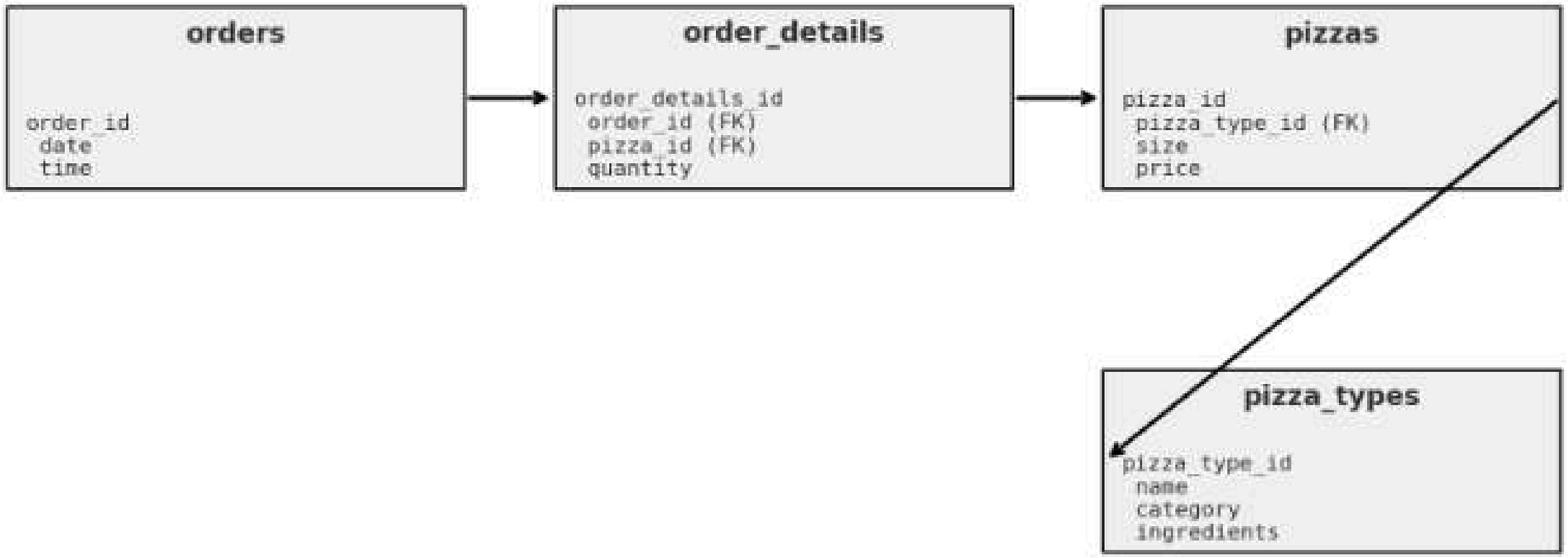
Hello . I’m Viraj Waikar, a data enthusiast passionate about using SQL and data to find real business insights.

In this project, I explored a Pizza Sales Dataset to simulate how a real restaurant might track and analyze their performance and I have utilies the SQL queries to solve the queries that were related to Pizza Sales

I used MySQL to clean, transform, and analyze the data, answering questions like:

- WHAT ARE THE TOP-SELLING PIZZAS?
- WHAT'S THE BEST-PERFORMING DAY OF THE WEEK?
- HOW DOES AVERAGE ORDER VALUE VARY?
- WHICH PIZZA SIZE SELLS THE MOST?

• Schema of Tables:



Retrieve the total number of orders placed

```sql
select count(order_id) as total_orders from orders;
```

total_orders
21350

Calculate the total revenue generated from pizza sales

```sql
select round(sum(order_details.quantity * pizzas.price), 2) as total_revenue
from order_details
join
pizzas on pizzas.pizza_id = order_details.pizza_id;
```

total_revenue
817860.05

Identify the highest-priced pizza


```sql
select pizza_types.name , pizzas.price
from pizza_types
join
pizzas on pizza_types.pizza_type_id = pizzas.pizza_type_id
order by pizzas.price desc limit 1;
```

	name	price
▶	The Greek Pizza	35.95

Identify the most common pizza size ordered


```
select pizzas.size, count(order_details.order_details_id) as order_count
from pizzas join order_details
on pizzas.pizza_id = order_details.pizza_id
group by pizzas.size;
```

size	order_count
M	15385
L	18526
S	14137
XL	544
XXL	28

List the top 5 most ordered pizza types along with their quantities


```sql
select pizza_types.name,
sum(order_details.quantity) as quantity
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id

join order_details
on order_details.pizza_id = pizzas.pizza_id
group by pizza_types.name order by quantity desc limit 5;
```

name	quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Join the necessary tables to find the total quantity of each pizza category ordered


```sql
select pizza_types.category,
sum(order_details.quantity) as quantity
from
pizza_types
join pizzas on pizza_types.pizza_type_id = pizzas.pizza_type_id
 join order_details
 on order_details.pizza_id = pizzas.pizza_id
 group by pizza_types.category
 order by quantity desc;
```

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Determine the distribution of orders by hour of the day


```sql
SELECT
    HOUR(order_time) AS Hour, COUNT(order_id) AS order_count
FROM
    orders
GROUP BY HOUR(order_time);
```

Hour	order_count
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642

Hour	order_count
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28
10	8
9	1

Join relevant tables to find the category-wise distribution of pizzas

```
SELECT
    category, COUNT(name)
FROM
    pizza_types
GROUP BY category;
```

category	COUNT(name)
Chicken	6
Classic	8
Supreme	9
Veggie	9

Group the orders by date and calculate the average number of pizzas ordered per day


```
select round(avg(quantity), 0 ) from
(select orders.order_date, sum(order_details.quantity) as quantity
from orders join order_details
on orders.order_id = order_details.order_id
group by orders.order_date) as order_quantity;
```

	round(avg(quantity), 0)
▶	138

Determine the top 3 most ordered pizza types based on revenue


```sql
select pizza_types.name,
sum(order_details.quantity * pizzas.price) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on order_details.pizza_id = pizzas.pizza_id
group by pizza_types.name order by revenue desc limit 3;
```

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

Calculate the percentage contribution of each pizza type to total revenue


```sql
select pizza_types.category,
round(sum(order_details.quantity * pizzas.price) / (select round(sum(order_details.quantity * pizzas.price), 2) as total_sales
from order_details
join pizzas on pizzas.pizza_id = order_details.pizza_id ) * 100 , 2) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on order_details.pizza_id = pizzas.pizza_id
group by pizza_types.category order by revenue desc;
```

category	revenue
Classic	26.91
Supreme	25.46
Chicken	23.96
Veggie	23.68

Determine the top 3 most ordered pizza types based on revenue for each pizza category


```sql
select  name, revenue
from
(select category, name, revenue,
rank() over (partition by category order by revenue desc) as rn
from
(select pizza_types.category, pizza_types.name,
sum(( order_details.quantity) * pizzas.price ) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on order_details.pizza_id = pizzas.pizza_id
group by  pizza_types.category, pizza_types.name) as a)  as b
where rn <= 3;
```

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5
The Classic Deluxe Pizza	38180.5

Analyze the cumulative revenue generated over time


```sql
select order_date ,
sum(revenue) over (order by order_date) cum_revenue
from
(select orders.order_date,
sum(order_details.quantity * pizzas.price) as revenue
from order_details join pizzas
on order_details.pizza_id = pizzas.pizza_id
join orders
on orders.order_id = order_details.order_id
group by orders.order_date ) as sales ;
```

order_date	cum_revenue
2015-01-01	2713.850000000004
2015-01-02	5445.75
2015-01-03	8108.15
2015-01-04	9863.6

order_date	cum_revenue
2015-12-24	807553.75
2015-12-26	809196.8
2015-12-27	810615.8
2015-12-28	812253

PROJECT SUMMARY/ KEY INSIGHTS

- 🍕 **Total number of pizzas sold: 21350**
- 📈**Highest selling pizza: The Thai Chicken Pizza**
- ⏰**Peak order time: 6:00 PM – 8:00 PM**
- ⏰ **Average Number of Pizzas ordered per Day: 138**
- 🍕 **Most ordered Pizza Size : L(Large) 18526**

CONCLUSION

- The project successfully used MySQL to analyze real-world pizza sales data.
- Schema was designed using normalized tables for orders, order details, and pizzas.
- Queries helped extract meaningful insights for business strategy.
- Analysis supports better inventory planning, marketing, and menu optimization.

LEARNINGS AND NEXT STEPS

Learned how to:

Design a schema in MySQL

Write and optimize SQL queries

Interpret sales data for business decisions

THANK YOU